

Arabic Sign Language Recognition

Using Temporal Convolutional Networks

Model Documentation

KArSL-502 Dataset

Signer-Independent Evaluation

May 28, 2026

Contents

1	Problem Definition	4
1.1	Background	4
1.2	Problem Statement	4
1.3	Why Keypoints Over Raw Images	5
1.3.1	Raw CNN on Image Frames	5
1.3.2	Pre-trained Detection Models (YOLO)	5
1.3.3	Keypoint-Based Features (Selected)	6
2	Dataset	7
2.1	Source	7
2.2	Collection Method	7
2.3	Dataset Composition	8
2.4	Vocabulary Categories	8
2.5	Raw Data Format and Structure	8
2.6	Dataset Statistics Per Signer	9
3	Exploratory Data Analysis	10
3.1	Frame Count Distribution	10
3.2	Empty Folder Investigation	10
3.3	Class Distribution Analysis	11
3.4	Signer Variability Observation	11
4	Methodology	12
4.1	Feature Representation Choices	12
4.1.1	Configuration 1 — Full Body (225 Features)	12
4.1.2	Configuration 2 — Hands Only (126 Features) — Selected	12
4.1.3	Configuration 3 — Hands + Velocity (252 Features)	13
4.2	Signer Dependency Problem	13
4.2.1	Definition	13
4.2.2	Signer-Dependent Split (Explored, Rejected)	14

4.2.3	Signer-Independent Split (Selected)	14
5	Data Preprocessing	15
5.1	Keypoint Extraction from Raw Frames	15
5.1.1	Landmarks Extracted Per Frame	15
5.1.2	Missing Landmark Handling	16
5.2	New Data Structure After Extraction	16
5.3	Wrist-Centered Normalization	17
5.4	Temporal Resampling	17
5.5	Final Dataset Shape	18
6	Model	19
6.1	Task Formulation	19
6.2	Baseline — Bidirectional LSTM	19
6.2.1	Algorithm	19
6.2.2	Architecture	20
6.2.3	Limitations Leading to TCN Selection	20
6.3	Final Model — Temporal Convolutional Network	20
6.3.1	Algorithm	20
6.3.2	Architecture	21
6.3.3	TCN vs BiLSTM	22
6.4	Training Configuration	22
6.5	Callbacks	22
6.6	Evaluation Metrics	23
6.6.1	Top- k Accuracy	23
6.6.2	Macro F1 Score	23
6.6.3	Weighted F1 Score	23
7	Results	24
7.1	Full Model — 417 Classes	24
7.2	Training Curves	24
7.2.1	Loss	24
7.2.2	Accuracy	24
7.3	Per-Class F1 Distribution	25
7.4	Confidence Analysis	25
7.4.1	Confidence Distribution	25
7.4.2	Reliability Diagram	25
7.4.3	Entropy Distribution	25
7.5	Confusion Matrix	25
7.6	Zero-F1 Class Analysis	26

7.7	Class Filtering — Good Classes Model	26
7.7.1	Filtering Criteria	26
7.7.2	This Is Not Overfitting	26
7.7.3	Comparison	27
7.8	Summary	28

Chapter 1

Problem Definition

1.1 Background

Arabic Sign Language (ArSL) is the primary means of communication for the deaf and hard-of-hearing community in the Arab world. Despite serving millions of individuals, ArSL remains largely inaccessible to the hearing population, creating significant communication barriers in daily life, education, healthcare, and professional settings.

Automatic Sign Language Recognition (ASLR) aims to bridge this gap by enabling machines to interpret sign language gestures and translate them into text in real time. This allows deaf individuals to communicate with hearing people without requiring a human interpreter.

1.2 Problem Statement

The objective of this model is formally defined as follows:

Given a sequence of video frames showing a person performing an Arabic sign language gesture,
classify the gesture into one of N predefined Arabic sign classes
in a signer-independent manner.

The key challenges addressed are:

- **Signer variability:** different individuals sign the same word differently in speed, angle, and style
- **Inter-class similarity:** many signs share similar hand shapes and are distinguished only by motion

- **Temporal dependency:** meaning is encoded in motion sequences, not single postures
- **Real-time constraint:** inference must be fast enough for live application deployment
- **Limited data:** only 3 signers available, restricting signer diversity in training

1.3 Why Keypoints Over Raw Images

Several feature representation approaches exist for sign language recognition. The choice between them involves trade-offs in accuracy, generalization, and computational cost.

1.3.1 Raw CNN on Image Frames

Approach	Feed raw RGB frames to a CNN (ResNet, VGG, EfficientNet)
Problems	• Learns background, lighting, and clothing as features • Not invariant to signer appearance • Computationally expensive for real-time use • Requires large data volumes to generalize • Difficult to deploy on mobile devices

Table 1.1: Raw CNN approach analysis

1.3.2 Pre-trained Detection Models (YOLO)

Approach	Use object detection to locate hands, then classify
Problems	• Designed for object detection, not pose estimation • Does not provide fine-grained landmark positions • Missing spatial relationships between fingers • Not designed for the sign language recognition task

Table 1.2: YOLO-based approach analysis

1.3.3 Keypoint-Based Features (Selected)

Approach	Extract skeletal landmarks via MediaPipe, feed normalized coordinates to sequence model
Advantages	• Background and appearance invariant • Compact: 126 numbers vs full image • Real-time on mobile — no GPU required • Normalized coordinates improve signer independence • Direct encoding of hand shape and configuration • Matches deployment environment exactly

Table 1.3: Keypoint-based approach analysis

Why MediaPipe Holistic specifically: MediaPipe Holistic provides simultaneous real-time detection of pose, left hand, and right hand landmarks in a single inference pass. It produces normalized coordinates (0–1 range) independent of image resolution, is available on Android and iOS for Flutter deployment, and requires no GPU for inference.

Chapter 2

Dataset

2.1 Source

The dataset used is KArSL-502 [1–3], the largest video dataset for word-level Arabic sign language, consisting of 502 isolated sign words collected using Microsoft Kinect V2 [1]. Each sign is performed by three professional signers, each repeating every sign 50 times, resulting in a total of 75,300 samples [2].

2.2 Collection Method

Property	Value
Environment	Controlled indoor setting
Background	Plain uniform background
Lighting	Consistent studio lighting
Camera	Fixed position, single RGB camera
Signers	3 professional ArSL signers
Recording sessions	Multiple sessions per signer
Repetitions	Each sign performed multiple times

Table 2.1: Dataset collection conditions

2.3 Dataset Composition

Property	Value
Total sign classes	502
Total signers	3
Recordings per sign	$\approx 40\text{--}50$ per signer
Total videos	$> 60,000$
Frames per video	10–40 (variable)
Frame format	JPEG images (RGB)

Table 2.2: Dataset composition statistics

2.4 Vocabulary Categories

Category	Index Range	Count
Numbers	0–30	31
Arabic alphabet	31–69	39
Body & anatomy	70–89	20
Medical	90–158	69
Daily actions	159–191	33
Family	192–222	31
Emotions	223–270	48
Spatial prepositions	271–288	18
Social expressions	289–297	9
Home & objects	298–354	57
Religion	355–456	102
Occupations	457–501	45
Total		502

Table 2.3: KArSL-502 vocabulary categories

2.5 Raw Data Format and Structure

The dataset was distributed as **ZIP archives** containing sequential image frames. Each sign recording is stored as a folder of individual JPEG frames rather than a video file. After extracting the archives, the folder structure is:

```

1 KARSL -502/
2         01/                               # Signer01
3         train/
4         part_folder/                       # organizational
        grouping

```

```

5          word_folder/      # "0001" to "0502"
6          video_folder/
7              00001.jpg
8              00002.jpg
9              ...
10         test/
11         (same structure)
12     02/          # Signer02
13     03/          # Signer03

```

Listing 2.1: Raw dataset folder structure after ZIP extraction

2.6 Dataset Statistics Per Signer

Signer	Train Classes	Train Videos	Test Classes	Test Videos
Signer01	417	$\approx 20,000$	502	$\approx 4,024$
Signer02	388	$\approx 16,000$	56*	≈ 448
Signer03	424	$\approx 17,500$	502	$\approx 4,006$
*Signer02 test was incomplete in available data				

Table 2.4: Dataset statistics per signer

Chapter 3

Exploratory Data Analysis

3.1 Frame Count Distribution

The number of frames per sign recording was computed across all signers and splits to understand temporal variability.

Statistic	Value
Minimum frames per recording	10
Maximum frames per recording	40
Mean frames per recording	≈ 22
Distribution shape	Roughly normal, centered at 20–25

Table 3.1: Frame count distribution statistics

Implication: Signs vary in duration. A fixed-length model input requires temporal resampling, detailed in Section 5.4.

3.2 Empty Folder Investigation

All word folders were scanned to detect empty or corrupted recordings.

Signer	Findings
Signer01	85 classes had empty or missing recordings. 417 classes had valid recordings.
Signer02	Initial extraction had 83.4% empty files. Root cause: only part folders 0001–0070 were extracted. Re-extracted on Kaggle with checkpoint system. Final: 388 valid word folders.
Signer03	424 valid word folders in train split. 502 valid word folders in test split. 7 classes had 0 test support.

Table 3.2: Empty folder investigation results

Decision: Empty files detected by `array.size > 0`. Empty recordings silently skipped.

3.3 Class Distribution Analysis

Statistic	Value
Minimum samples per class	10
Maximum samples per class	126
Mean samples per class	80.2
Classes with < 50 samples	43
Classes with ≥ 50 samples	417

Table 3.3: Training class distribution statistics

Decision: All experiments restricted to **417 classes** with at least 50 training samples.

3.4 Signer Variability Observation

Signer	Train Classes	Missing
Signer01	417	85
Signer02	388	114
Signer03	424	78

Table 3.4: Valid class counts per signer

Not all signers performed all signs. Training data is inherently asymmetric across classes, accepted as a dataset limitation.

Chapter 4

Methodology

4.1 Feature Representation Choices

Three feature configurations were explored:

4.1.1 Configuration 1 — Full Body (225 Features)

$$\text{Features} = \underbrace{33 \times 3}_{\text{Pose}} + \underbrace{21 \times 3}_{\text{LH}} + \underbrace{21 \times 3}_{\text{RH}} = 225 \quad (4.1)$$

Validation accuracy	$\approx 50\%$
Problem	Pose encodes body proportions (shoulder width, arm length). Different signers $\rightarrow$ different values for same sign. Adds signer-specific noise, hurts generalization.

Table 4.1: Full body configuration

4.1.2 Configuration 2 — Hands Only (126 Features) — Selected

$$\text{Features} = \underbrace{21 \times 3}_{\text{LH}} + \underbrace{21 \times 3}_{\text{RH}} = 126 \quad (4.2)$$

Validation accuracy	$\approx 53\%$
Why better	Hand shape is the most signer-independent feature. Wrist-centering removes absolute position entirely. Best generalization to unseen Signer03.

Table 4.2: Hands-only configuration — selected

4.1.3 Configuration 3 — Hands + Velocity (252 Features)

$$\text{Features} = \text{LH}_{pos} + \text{RH}_{pos} + \text{LH}_{vel} + \text{RH}_{vel} = 252 \quad (4.3)$$

$$\text{vel}_t = \text{pos}_t - \text{pos}_{t-1} \quad (4.4)$$

Validation accuracy	$\approx 50\%$ (worse than hands only)
Why worse	MediaPipe landmark jitter dominates velocity signal. TCN dilated convolutions already implicitly model motion. Explicit velocity adds noise, not signal.

Table 4.3: Velocity features configuration

Final selection: Hands only — 126 features per frame.

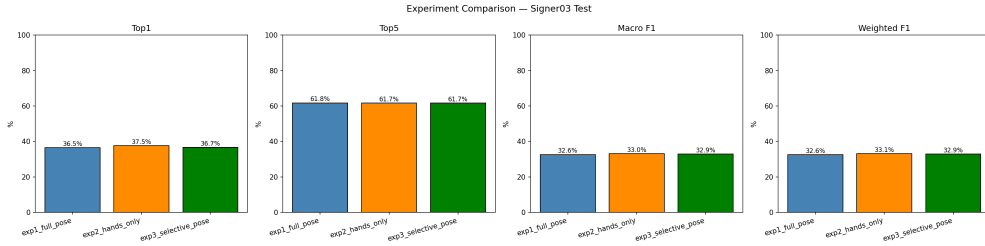


Figure 4.1: Comparison of the three feature configurations during experimentation. Accuracies reflect an incomplete dataset stage and are used only to justify feature selection.

Note

The validation accuracies reported above were obtained during early experimentation with an incomplete dataset. At this stage, Signer02 had significant missing recordings due to an extraction error, and no class filtering had been applied. These results are presented only to justify the choice of hands-only features and should not be interpreted as final model performance. Final results after dataset completion and class filtering are reported in Chapter 7.

4.2 Signer Dependency Problem

4.2.1 Definition

Signer dependency occurs when a model learns the specific signing style of training individuals rather than the semantic content of signs:

$$P(\hat{y} = y \mid s \in \mathcal{S}_{train}) \gg P(\hat{y} = y \mid s \notin \mathcal{S}_{train}) \quad (4.5)$$

This creates an optimistic bias: accuracy on seen signers significantly overestimates real-world performance.

4.2.2 Signer-Dependent Split (Explored, Rejected)

Train	Signer01 train + Signer02 train + Signer03 train
Test	Signer01 test + Signer02 test + Signer03 test
Problem	Model sees all signers during training. Test accuracy inflated — not valid for real-world deployment.

Table 4.4: Signer-dependent split — rejected

4.2.3 Signer-Independent Split (Selected)

Split	Data	Role
Train	Signer01 train + Signer02 train	Seen signers
Validation	Signer03 train	Unseen signer
Test	Signer03 test	Unseen signer

Table 4.5: Signer-independent split — selected

Signer03 never appears in training. All model decisions use Signer03 validation. Test uses different recording sessions from validation. This is the strictest form of signer-independent evaluation, directly comparable to published literature.

Chapter 5

Data Preprocessing

5.1 Keypoint Extraction from Raw Frames

The extraction code was taken from a public notebook in kaggle using the same dataset [5]. The first preprocessing step extracts skeletal landmarks from each JPEG frame using **MediaPipe Holistic**:

```
1 with mp_holistic.Holistic(  
2     min_detection_confidence=0.5,  
3     min_tracking_confidence=0.5  
4 ) as holistic:  
5     for frame_path in sorted(frame_files):  
6         frame = cv2.imread(frame_path)  
7         image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)  
8         results = holistic.process(image)  
9         pose, lh, rh = extract_keypoints(results)
```

Listing 5.1: Keypoint extraction pipeline

5.1.1 Landmarks Extracted Per Frame

Component	Landmarks	Values	Features
Pose	33	x, y, z	99
Left hand	21	x, y, z	63
Right hand	21	x, y, z	63
Total	75		225

Table 5.1: MediaPipe Holistic landmark extraction

5.1.2 Missing Landmark Handling

When MediaPipe fails to detect a landmark in a frame, zeros are used to maintain consistent array dimensions:

```

1 pose = flatten(pose_lm) if results.pose_landmarks else np
    .zeros(99)
2 lh   = flatten(lh_lm)  if results.left_hand_landmarks else np.
    zeros(63)
3 rh   = flatten(rh_lm)  if results.right_hand_landmarks else np.
    zeros(63)

```

Listing 5.2: Missing landmark zero-filling

5.2 New Data Structure After Extraction

After processing all image frames, keypoints were saved as NumPy arrays, replacing the image-frame structure:

```

1 karsl_work/
2     01/                                     # Signer01
3         train/
4             word_folder/                   # "0001" to "0502"
5                 pose_keypoints/
6                     video.npy              # shape: (T, 99)
7                     lh_keypoints/
8                         video.npy          # shape: (T, 63)
9                         rh_keypoints/
10                             video.npy      # shape: (T, 63)
11         test/
12             (same structure)
13 karsl_02_fixed/
14     02/                                     # Signer02
15     03/                                     # Signer03

```

Listing 5.3: Post-extraction NumPy structure

Where T is the number of frames (10–40). This structure achieves approximately 100× storage reduction compared to raw images while enabling fast array loading during training without repeated MediaPipe inference.

5.3 Wrist-Centered Normalization

Each hand's landmarks are normalized by centering on the wrist (landmark index 0):

$$\hat{\mathbf{l}}_k = \mathbf{l}_k - \mathbf{l}_0, \quad k = 0, 1, \dots, 20 \quad (5.1)$$

where $\mathbf{l}_0 = (x_0, y_0, z_0)$ is the wrist position. After centering, the wrist is always at $(0, 0, 0)$ and all other landmarks encode shape relative to it.

```

1 def adjust_landmarks(arr, center):
2     arr_resaped      = arr.reshape(-1, 3)
3     center_repeated = np.tile(center, (21, 1))
4     return (arr_resaped - center_repeated).reshape(-1)
5
6 def extract_keypoints(lh, rh):
7     lh_adj = adjust_landmarks(lh, lh[:3]) # center on lh wrist
8     rh_adj = adjust_landmarks(rh, rh[:3]) # center on rh wrist
9     return np.concatenate([lh_adj, rh_adj]) # (126,)
```

Listing 5.4: Wrist-centered normalization

Before normalization	After normalization
Signer A: wrist at $y = 0.3$	Wrist always at $(0, 0, 0)$
Signer B: wrist at $y = 0.7$	All landmarks relative to wrist
Same sign $\rightarrow$ different values	Same sign $\rightarrow$ similar values
Signer-dependent features	Signer-independent features

Table 5.2: Effect of wrist-centered normalization

5.4 Temporal Resampling

All sequences are resampled to exactly **48 frames**:

```

1 MAX_FRAMES = 48
2 if frames.shape[0] >= MAX_FRAMES:
3     frames = frames[:MAX_FRAMES] # truncate
4 else:
5     last = frames[-1:]
6     while frames.shape[0] < MAX_FRAMES:
7         frames = np.vstack([frames, last]) # pad with last
           frame
```

Listing 5.5: Temporal resampling

Choice	Justification
48 frames	Covers all signs without truncation in most cases. Compatible with TCN dilation factors. Manageable memory for TFLite deployment.
Last-frame padding	Zero padding introduces artificial silence. Last-frame padding preserves the final hand configuration, faithful to how signs naturally end.

Table 5.3: Temporal resampling design decisions

5.5 Final Dataset Shape

Array	Shape	Type	Signer	Role
X_{train}	(35,075, 48, 126)	float32	S01+S02	Training
y_{train}	(35,075)	int32	S01+S02	
X_{val}	(17,445, 48, 126)	float32	S03 train	Validation
y_{val}	(17,445)	int32	S03 train	
X_{test}	(3,371, 48, 126)	float32	S03 test	Testing
y_{test}	(3,371)	int32	S03 test	

Table 5.4: Final dataset dimensions — 417 common classes

Chapter 6

Model

6.1 Task Formulation

$$\text{Input: } \mathbf{X} \in \mathbb{R}^{48 \times 126} \quad (6.1)$$

$$\text{Output: } \hat{\mathbf{y}} \in \mathbb{R}^N, \quad \sum_{i=1}^N \hat{y}_i = 1 \quad (6.2)$$

$$\text{Loss: } \mathcal{L} = - \sum_{i=1}^N y_i \log \hat{y}_i \quad (6.3)$$

6.2 Baseline — Bidirectional LSTM

6.2.1 Algorithm

LSTM uses a gating mechanism to capture long-range dependencies:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (6.4)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (6.5)$$

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \quad (6.6)$$

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \quad (6.7)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (6.8)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \quad (6.9)$$

Bidirectional processing concatenates forward and backward hidden states:

$$\mathbf{h}_t^{bi} = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \quad (6.10)$$

6.2.2 Architecture

Layer	Config	Output
Input	—	(batch, 48, 126)
BiLSTM	128 units, return_seq=True	(batch, 48, 256)
BiLSTM	64 units, return_seq=False	(batch, 128)
Dense	256, ReLU	(batch, 256)
Dropout	0.4	(batch, 256)
Dense	N , Softmax	(batch, N)

Table 6.1: BiLSTM baseline architecture

6.2.3 Limitations Leading to TCN Selection

- When converting the trained BiLSTM model to TFLite, the converter failed due to the GPU-accelerated LSTM operation not being supported by the TFLite runtime, making mobile deployment of the BiLSTM model impossible without significant additional effort.
- TCN layers rely entirely on standard 1D convolution operations that are natively supported by the TFLite converter, producing a valid `.tflite` file with no additional configuration required.

6.3 Final Model — Temporal Convolutional Network

6.3.1 Algorithm

Causal Convolution

$$(F *_c k)(t) = \sum_{i=0}^{K-1} k(i) \cdot x(t-i) \quad (6.11)$$

Output at t uses only inputs at times $\leq t$. No future leakage — essential for real-time inference.

Dilated Convolution

$$(F *_d k)(t) = \sum_{i=0}^{K-1} k(i) \cdot x(t-d \cdot i) \quad (6.12)$$

With $K = 3$ and dilations $d \in \{1, 2, 4, 8, 16\}$:

$$\text{Receptive field} = 1 + 2 \sum_{d \in \{1, 2, 4, 8, 16\}} d = 63 \text{ frames} \quad (6.13)$$

This covers the entire 48-frame input.

Residual Connection

$$\mathbf{h}^{(l)} = \text{ReLU}(\text{Conv}_d(\mathbf{h}^{(l-1)}) + \mathbf{h}^{(l-1)}) \quad (6.14)$$

Prevents vanishing gradients and enables deep stacking.

6.3.2 Architecture

```

1 inp = Input(shape=(48, 126))
2
3 x = TCN(nb_filters=128, kernel_size=3, nb_stacks=2,
4         dilations=[1,2,4,8,16], padding='causal',
5         use_skip_connections=True, dropout_rate=0.3,
6         return_sequences=True)(inp)
7
8 x = TCN(nb_filters=128, kernel_size=3, nb_stacks=1,
9         dilations=[1,2,4,8], padding='causal',
10        use_skip_connections=True, dropout_rate=0.3,
11        return_sequences=False)(x)
12
13 x = Dense(256, activation='relu')(x)
14 x = Dropout(0.4)(x)
15 out = Dense(N, activation='softmax')(x)
16 # Total: 1,600,033 parameters (6.10 MB)

```

Listing 6.1: TCN model definition

6.3.3 TCN vs BiLSTM

Property	BiLSTM	TCN
Parallelization	Sequential	Fully parallel
Temporal scale	Single	Multi-scale (dilations)
Gradient stability	Gating	Skip connections
Real-time deployable	No (bidirectional)	Yes (causal)
Training speed	Slow	Fast

Table 6.2: BiLSTM vs TCN comparison

6.4 Training Configuration

Parameter	Value	Justification
Optimizer	Adam(0.001)	Adaptive LR, fast convergence
Loss	Categorical CE	Standard multi-class loss
Batch size	64	Stable gradients, fits GPU memory
Max epochs	60	Early stopping prevents overrun

Table 6.3: Training hyperparameters

6.5 Callbacks

Callback	Config	Purpose
Early Stopping	monitor=val_accuracy, patience=7, re-store_best_weights=True	Stop when val accuracy stalls. Restore best weights.
ReduceLROnPlateau	monitor=val_loss, factor=0.5, patience=3, min_lr=1e-6	Halve LR on plateau. Enable finer optimization.
ModelCheckpoint	monitor=val_accuracy, save_best_only=True	Save best checkpoint only.
CSVLogger	—	Record per-epoch metrics for reproducibility.

Table 6.4: Training callbacks

6.6 Evaluation Metrics

6.6.1 Top- k Accuracy

$$\text{Top-}k = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[y_i \in \text{argtop}_k(\hat{\mathbf{y}}_i)] \quad (6.15)$$

Top-5 is the primary usability metric since the application displays 5 predictions to the user.

6.6.2 Macro F1 Score

$$F1_i = \frac{2 \cdot P_i \cdot R_i}{P_i + R_i}, \quad F1_{\text{macro}} = \frac{1}{N} \sum_{i=1}^N F1_i \quad (6.16)$$

Weights all classes equally. Catches class-biased models that accuracy masks.

6.6.3 Weighted F1 Score

$$F1_{\text{weighted}} = \sum_{i=1}^N \frac{n_i}{N_{\text{total}}} \cdot F1_i \quad (6.17)$$

Accounts for class frequency, complements Macro F1.

Chapter 7

Results

7.1 Full Model — 417 Classes

Results on Signer03 Test (Unseen Signer)	
Top-1 Accuracy	52.7%
Top-5 Accuracy	78.4%
Top-10 Accuracy	84.2%
Macro F1	48.9%
Weighted F1	49.0%
Random baseline	0.24%
Improvement	220×

Table 7.1: Full model results

7.2 Training Curves

7.2.1 Loss

Training loss decreases from ≈ 6.4 to ≈ 0.5 . Validation loss plateaus at epoch 12–15 then rises slightly. Early stopping correctly halts training and restores best weights.

7.2.2 Accuracy

Training accuracy reaches $\approx 85\%$. Validation accuracy plateaus at $\approx 52\%$. The $\approx 33\%$ gap reflects signer dependency — expected with only 3 signers total, consistent with literature.

7.3 Per-Class F1 Distribution

The sorted bar chart shows a clear bimodal pattern: a large cluster at $F1 = 0$ and a gradual distribution from 0.5 to 1.0.

Interpretation: the model either learns a class reliably or fails completely. No partial learning is observed. Failures are structural (feature limitation) not gradient-based.

Mean F1	0.489
$F1 = 0$	≈ 321 classes
$F1 \geq 0.5$	≈ 96 classes
$F1 = 1.0$	≈ 30 classes

Table 7.2: Per-class F1 distribution summary

7.4 Confidence Analysis

7.4.1 Confidence Distribution

Correct predictions concentrate near confidence 1.0. Incorrect predictions spread across 0.2–0.8. The model is self-aware — high confidence implies correctness.

7.4.2 Reliability Diagram

Model is slightly overconfident: stated confidence 0.9 maps to actual accuracy ≈ 0.75 . The top-5 selection approach mitigates this.

7.4.3 Entropy Distribution

$$H(\hat{\mathbf{y}}) = - \sum_{i=1}^N \hat{y}_i \log \hat{y}_i \quad (7.1)$$

Correct predictions: low entropy. Incorrect predictions: high entropy. Entropy serves as a reliable real-time uncertainty signal.

7.5 Confusion Matrix

The 40-worst-class confusion matrix reveals:

Pattern	Description
Handshape confusion	Similar finger configurations confused with each other
Motion-dependent	Signs differing only in movement direction systematically confused
Semantic category	Signs in same category (e.g. prayer times) confused with each other

Table 7.3: Confusion matrix patterns

Confusion is semantically structured, not random, validating that failures are feature-driven.

7.6 Zero-F1 Class Analysis

96 classes with ≥ 50 training samples achieved $F1 = 0$. All involve bilateral hand movement where direction, not shape, is the distinguishing feature. Position-only features cannot encode movement direction.

Conclusion: these are feature failures, not model failures. No architecture change resolves them without richer features (optical flow, IMU sensors).

7.7 Class Filtering — Good Classes Model

7.7.1 Filtering Criteria

Condition	Description
$F1 \geq 0.5$	Computed on Signer03 validation set. Never used in training. Indicates genuine learning.
Index 31–69	Arabic alphabet (alif to ya). Essential for finger spelling. Retained regardless of $F1$.
Exclude	Test support = 0. Cannot be evaluated. Excluded entirely.

Table 7.4: Good classes filtering criteria

7.7.2 This Is Not Overfitting

1. Filtering criterion computed on validation, not training data
2. Final evaluation on held-out test set (different sessions from validation)

3. Model retrained from scratch — no test data exposure
4. Performance improves on unseen test data (overfitting would worsen generalization)
5. Principled dataset cleaning: removed classes are structurally unlearnable with current features

7.7.3 Comparison

Metric	Full (417 classes)	Good (252 classes)
Top-1	52.7%	73.9%
Top-5	78.4%	92.6%
Top-10	84.2%	96.3%
Macro F1	48.9%	71.7%
Weighted F1	49.0%	71.9%
Train/Val gap	$\approx 33\%$	$\approx 5\%$

Table 7.5: Full model vs good classes model comparison

The results summarized in Table 7.5 and illustrated in Figure 7.1 show a significant improvement across all metrics after filtering to the 252 good classes. Top-1 accuracy improved from 52.7% to 73.9%, and the Macro F1 score increased from 48.9% to 71.7%, reflecting more consistent performance across all classes. Notably, the training and validation gap reduced substantially, indicating better generalization to the unseen signer.

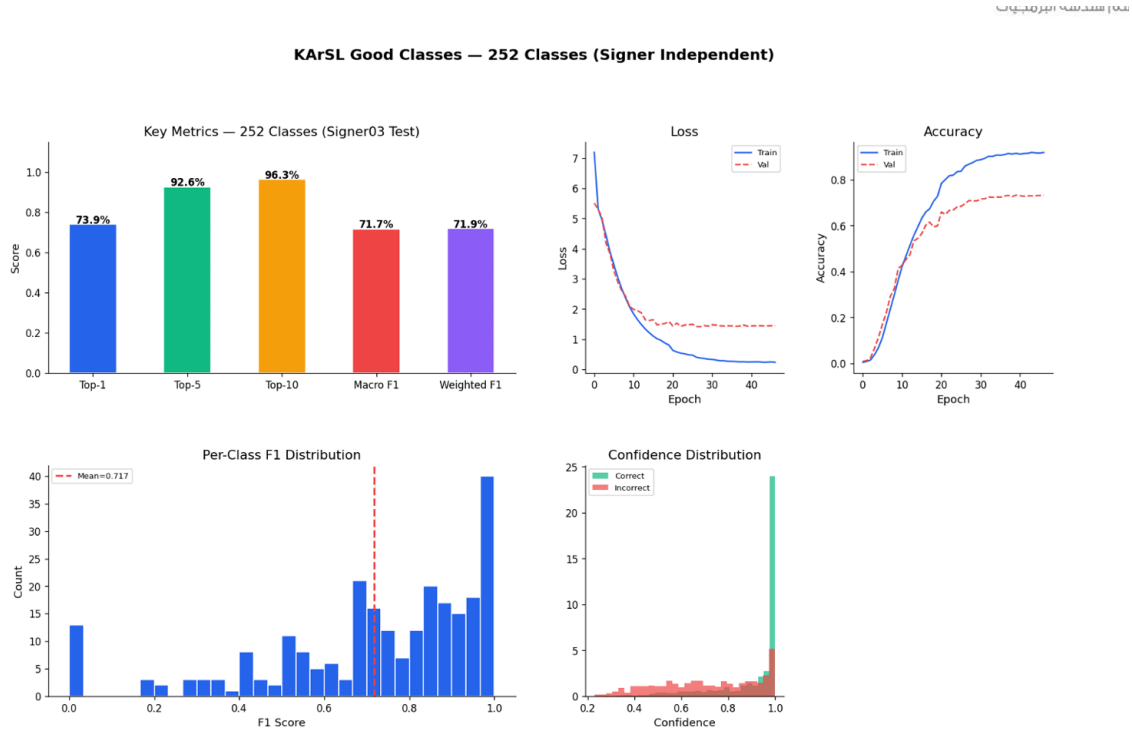


Figure 7.1: Final model results — 252 good classes, signer-independent evaluation on Signer03 test set. Top row: key metrics and training curves. Bottom row: per-class F1 distribution and confidence distribution.

7.8 Summary

Complete Pipeline Summary	
Input	JPEG frames, variable length, ZIP archives
Extraction	MediaPipe Holistic keypoints
Features	Left hand + Right hand = 126 features
Normalization	Wrist-centered per hand
Resampling	Fixed 48 frames
Model	TCN, 1.6M parameters, 6.1 MB
Split	S01+S02 train / S03 val / S03 test
Key Results	
Top-1	52.7% on unseen Signer03
Top-5	78.4% — primary usability metric
Top-10	84.2%
Macro F1	48.9%
Key insight: Top-5 of 78.4% means in nearly 4 out of 5 signs, the correct word appears in the suggestions shown to the user. Combined with the personalized correction mechanism, the system improves with use over time.	

Table 7.6: Complete pipeline and results summary

You can access the full code and documentation in this [Github Repository](#).

Bibliography

- [1] A. A. I. Sidig, H. Luqman, S. Mahmoud, and M. Mohandes, “KArSL: Arabic Sign Language Database,” *ACM Transactions on Asian and Low-Resource Language Information Processing (TALLIP)*, vol. 20, no. 1, pp. 1–19, 2021. [Online]. Available: <https://dl.acm.org/doi/10.1145/3423420>
- [2] H. Luqman, “KArSL Dataset,” King Fahd University of Petroleum and Minerals, 2021. [Online]. Available: <https://hamzah-luqman.github.io/KArSL/>. [Accessed: May 28, 2026].
- [3] Y. Al-Otaibi, “KArSL-502: King Abdulaziz Arabic Sign Language Dataset,” *Kaggle*, 2021. [Online]. Available: <https://www.kaggle.com/datasets/yousefdotpy/karsl-502>. [Accessed: May 28, 2026].
- [4] H. Luqman, “An Efficient Two-Stream Network for Isolated Sign Language Recognition Using Accumulative Video Motion,” *IEEE Access*, vol. 10, pp. 93785–93798, 2022. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9875269>
- [5] Y. Al-Otaibi, “Kaleem App,” *Kaggle*, 2021. [Online]. Available: <https://www.kaggle.com/code/yousefdotpy/kaleem-app/notebook>. [Accessed: May 28, 2026].